

AI 使用说明情况

SGLang 在线推理服务挑战 · AI 协助的范围、方式与可靠性核查

一、使用了哪些模型与工具

环节	所用 AI / 工具	承担的具体工作
主体执行	WorkBuddy 智能体（具备本地文件读写与命令执行能力的 LLM 代理）	阅读任务 PDF；探测硬件与网络；编写 WSL 端全套脚本；驱动「编译 → 起服务 → 验证 → 压测」全流程；生成日志、指标与交付文档
资料核对	大语言模型 + 联网检索	核对 SGLang 0.5.14 的 CPU 构建配方、Mooncake trace 字段结构、两篇论文（SGLang NeurIPS'24 / vLLM SOSP'23）的官方摘要与实验数据
人工部分	本人	提供 WSL2 与 Ubuntu 环境；解除本机安全策略对 <code>wsl.exe</code> 的拦截；确认最终交付形态与取舍

二、我实际使用的提示词

我并没有反复雕琢提示词，主要用了一条“把主动权交出去”的指令，后续全部在同一条对话里追问。

初始提示词

```
@first-challenge.pdf 帮我完成任务，基本由你来完成，我安装了 wsl2，wsl 里有 ubuntu，里面配置了 git 和 github
```

过程中追加的关键提示

```
wsl 好了
```

第二条只有三个字，但它是整条链路能跑通的前提 —— 在此之前 `wsl.exe` 被本机安全策略列入程序黑名单，AI 无法在 WSL 内执行任何命令，只能先把不依赖 WSL 的交付物做完并等待。这也说明：**当 AI 的执行能力被环境卡住时，最有价值的动作不是换提示词，而是先把环境解锁。**

三、AI 具体是怎么帮到学习的

- 把“黑盒报错”变成“可验证的假设链”。在 `Illegal instruction` 这个最棘手的问题上，AI 没有停在猜测，而是按“单独 import 定位模块 → `objdump` 反汇编数出 98672 处 `zmm` 引用 → `/proc/cpuinfo` 确认无 AVX-512 → 回到 CMakeLists 找到硬编码的 `-march=x86-64-v4` → 在 `vec.h` 里找到 `CPU_CAPABILITY_AVX512` 的宏守卫与通用回退分支”的顺序逐层取证。这条链本身就是一次很好的调试示范，我照着复现了一遍，学会了怎么用二进制与硬件层面的证据去回答上层语言回答不了的问题。
- 提前消灭不确定性。它先识别出 Watt Toolkit 的 GitHub 加速在做 TLS 中间人（`git clone` 直接失败），于是把 Mooncake trace、SGLang 源码包、CPython、以及 1.5 GB 的模型权重 **全部预取到本地**，让 WSL 侧完全离线可构建。这个“先做减法再动手”的顺序，比一路 debug 网络问题高效得多。
- 用数据支撑设计决策。压测前先统计 trace：23608 条记录中 `hash_ids[0]` 只有 4 个取值、最常见占 46.3%，据此设计“所有请求共享 256 token 前缀”，随后服务端日志出现 76.3% 的 `prefill` 批次命中 RadixCache。这让抽象概念（前缀复用）第一次变成了我能亲眼看到的数字。
- 把一切落成可复现的脚本。最终交付包含完整脚本与原始日志，而不是“我这边跑通了”的口头结论 —— 这让结果可以被别人验证，也更接近真实工程的习惯。

四、AI 是否误导过我？—— 如实说明

有过，而且不止一次。我认为把这些记下来比只写优点更有价值。

AI 的判断	是否正确	实际情况
按官方 <code>xeen.Dockerfile</code> 的 CPU 配方编译 SGLang，即可在 CPU 机器上运行	部分错误	编译确实成功，但无法运行。官方“CPU 配方”隐含要求 AVX-512 + AMX，文档并未明说。教训：官方文档的“支持 CPU”不等于“支持任何 CPU”。
进度卡住是因为安全策略拦截 <code>wsl.exe</code> ，需要用户解锁	正确	这正是真实阻塞点；AI 没有假装能绕过，而是明确说明无法从自身侧解除。
把 <code>-march=x86-64-v4</code> 降为 <code>x86-64-v3</code> 后可以正常运行	正确	已实测通过。AI 在动手前先用 <code>vec.h</code> 的宏守卫与 <code>#else</code> 回退分支验证了可行性，而不是盲目试错。
脚本里 <code>MODEL_ID="..."#</code> 注释 是合法写法	错误	少一个空格导致 bash 把整行解析为“赋值 + 命令”，变量根本没被赋值，脚本以 <code>unbound variable</code> 退出。这是 AI 自己写出的 bug，由运行日志暴露后被修正。

我的结论：AI 在“检索、生成脚本、按证据推进排查”上非常强，但它的输出**必须被真实运行结果检验**。上面第一行就是典型 —— 编译成功这种“看起来对了”的信号极具迷惑性，只有真正 `import` 一次、真正发出一条推理请求，才能证伪。所以我的用法是：让 AI 负责提出假设和写代码，让日志和实测数字负责审判。

五、AI 使用的边界声明

- AI 全程只在本机 WSL 内操作，未访问任何外部账号或服务；
- 所有日志、指标、截图均取自真实运行过程，未做人工改写或美化；
- 对 AI 给出的技术结论（如论文数据、构建配方），我均以官方来源交叉核对过；
- 最终的环境取舍（如压测长度上限、AVX2 基线的性能代价）由我确认并已如实写入交付说明。

—— 以上为本次作业中 AI 使用的完整说明。